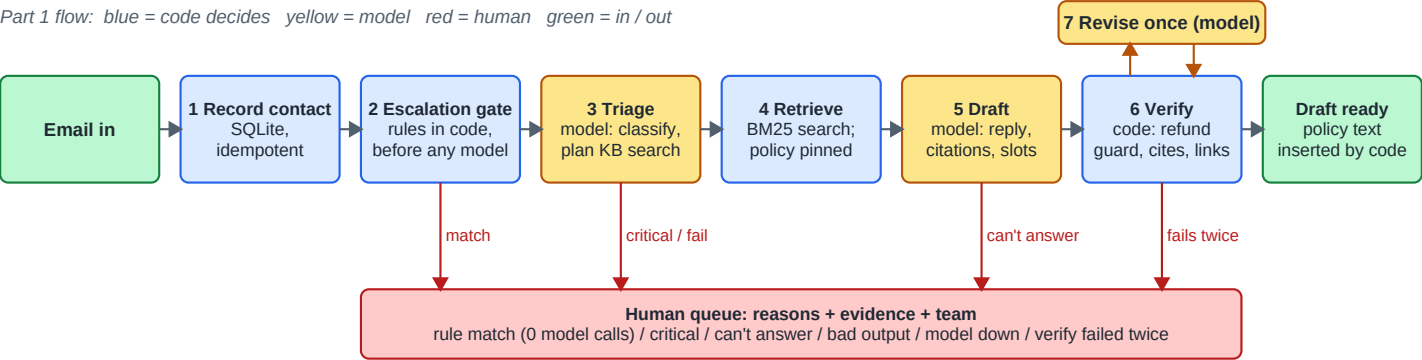


Agentic Architect Challenge: architecture on one page

Python 3.10+, one shared toolkit (LLM clients for Gemini and OpenAI behind one interface, retries, circuit breaker, JSON validation, trace-id logging) and three parts. Everything runs offline with a fake model; the same code runs live with a key in .env.

Part 1 flow: blue = code decides yellow = model red = human green = in / out



Principles

- **The model plans and writes; code decides and verifies.** Escalation, refund facts, summary length and calculator safety are deterministic code with unit tests.
- **Fail closed:** uncertain, invalid or unavailable means a human handles the email.
- **The model can only escalate up** (critical flag, "not answerable"), never remove a rule match.
- **One trace id per request** on every JSON log line; no key, prompts or email bodies in logs.

Part 1: support-email agent

- **Escalation gate** before any model call: data loss / outage / breach (regex on NFKC-normalised text), **more than 3 contacts in 7 days** (current email counts, window (t-7d, t], so the 4th escalates), injection phrases. Escalated emails cost 0 model calls and still route during outages.
- **Multi-label triage** (Billing, Technical, Feedback, Other) validated against a fixed list; the model also writes the KB search queries (plan), code runs BM25 (act).
- **No invented refund facts:** the model may only write `{{policy:R2}}`; code rejects refund claims in its own words and inserts the exact clause text. Check fails twice: human.

Part 2: scraper fix

Bottleneck: the whole raw page in one prompt. **Fix:** safe fetch (timeouts + deadline, size cap, content type, SSRF, robots) -> main-content extraction (-41% to -66% text on live pages) -> Playwright for JS-only pages -> map-reduce (max 10 map calls, max 40k chars each) -> **word limit measured in code:** ask once to compress, then cut at a sentence. Never over the limit.

Part 3: document agent

Hand-written tool loop over raw Gemini/OpenAI calls (max 5 steps, stops on repeated calls). Memory: token-budgeted window of whole turns + pinned user name. Calculator: AST whitelist, never eval; the model decides when to call it (measured by a 6-question eval).

Trade-offs

Choice	Why	Cost we accept
Regex gate, not the model	deterministic, injection-proof, works when the model is down	misses paraphrases; escalates negations
Policy placeholders	a misquoted policy is impossible	formal tone; wrong clause choice possible; strict checker = more human reviews
BM25, not embeddings	no extra service, free, deterministic	word match only (helped by model-written queries)
Raw API + own loop	every line explainable; 5 dependencies	no built-in checkpointing / graph UI
Threads + SQLite	simple; model calls are I/O-bound	single node; needs a queue at scale

Failure points and outcomes

Failure	Handling
Model down / 5xx / 429 / timeout	3 attempts, jittered backoff, honours Retry-After; circuit breaker; RPM limiter; then human queue
Malformed or invalid JSON	validated in code; one repair with the exact error; then human
Invented refund term, citation or link	verifier rejects; one revision; then human
Prompt injection (email, page, document)	data-only tags; safety in code; hidden page text dropped; only tool is side-effect free
Runaway tool loop	5-step limit + repeated-call stop; turn rolled back cleanly
Slow / huge / non-HTML / JS-only page	deadline, 5 MB cap, content-type check, Playwright render
Duplicate email, parallel workers	idempotent message id; counts by timestamp
Bug in our code	batch catch-all: human (internal_error), stack trace logged

At production scale

Queue + stateless workers, Postgres/Redis keyed by CRM id, hybrid retrieval, review UI, labelled eval set in CI, OpenTelemetry export of the existing spans, second-provider fallback.

Verified: 215 offline tests (fake model + mocked HTTP; no key, no network), including Playwright rendering on a local JS page; offline load test (500 emails, 10% injected outages, every email routed); fetch/extract/render on live pages (Wikipedia, python.org, a JS-only page). **Live Gemini runs:** see README. **Not verified live:** the OpenAI client (mocked HTTP only).